

NETRA

Unified CCTV Integration and Intelligence Platform

High-Level Design, Scalability, API and Requirements Traceability

Gujarat Police Innovation Challenge 2026

High-Level Design

High-Level Design (HLD) - Unified CCTV Integration & Video Analytics Platform

Project: Netra - Statewide CCTV Integration & Intelligence Platform **Submitted for:** Gujarat Police Innovation Challenge 2026 **Chosen approach:** Hybrid - **Model 1 (mandatory)** + **Model 2 (Unified Viewing & Metadata Analytics)** + **Model 3 (VMS Federation & Middleware)**, with selective central AI analytics **Version:** 0.2 (as-built + production design) | **Date:** 2026-08-17

> **Status:** A working prototype of this design has been built and runs against > the **real** Gujarat CSITMS live feed (live.corp8.cloud, currently 30 cameras). > Sections below marked **[built]** describe what is implemented; the rest is the > production design that the prototype's interfaces are ready to scale into. A > summary of exactly what was implemented (and its honest limitations) is in Section 19.

1. Executive summary

Gujarat's 26 departments operate independent, heterogeneous CCTV systems - mixed analog/IP cameras, multiple VMS vendors, varied storage and retention, dispersed up to ~1,000 km. Netra unifies them **without ripping out or replacing any existing system**, using a vendor-neutral federation layer that leaves departmental infrastructure intact while delivering a single pane of glass for viewing, analytics, and real-time alerting.

The design is driven by one hard fact (see [SCALABILITY.md](#)): at 80,000 cameras, centralizing video is ~200 Gbps and ~32 PB - infeasible - while the *events* extracted from that video are ~0.3% of the data. **Netra is therefore event-driven and hierarchical:** analytics run at the edge/region, video stays near its source, and only compact events + on-demand streams traverse the network. This single principle answers heterogeneity, geographic dispersion, and scalability together.

The platform delivers the challenge's scored test case - onboard ~50 feeds and, given a vehicle number, reconstruct its **cross-camera route with a timestamped, location-wise movement history** - as an indexed search over events, not a video-crunching problem.

2. Objectives & scope

In scope: unified registry + GIS visibility (Model 1); federation of departmental VMS/cameras via adapters (Model 3); central/edge AI analytics - ANPR, vehicle/person/object detection, cross-camera vehicle tracking (Model 4); integration with VAHAN, SARTHI, eGujCop (CCTNS), AFIS, NAFIS for real-time alerts; command center; statewide scalability to ~80,000 cameras; viewing support for permitted private cameras (societies/malls).

Out of scope (by design): replacing departmental VMS/storage; centralizing all raw video; owning departmental retention policy.

3. Design principles

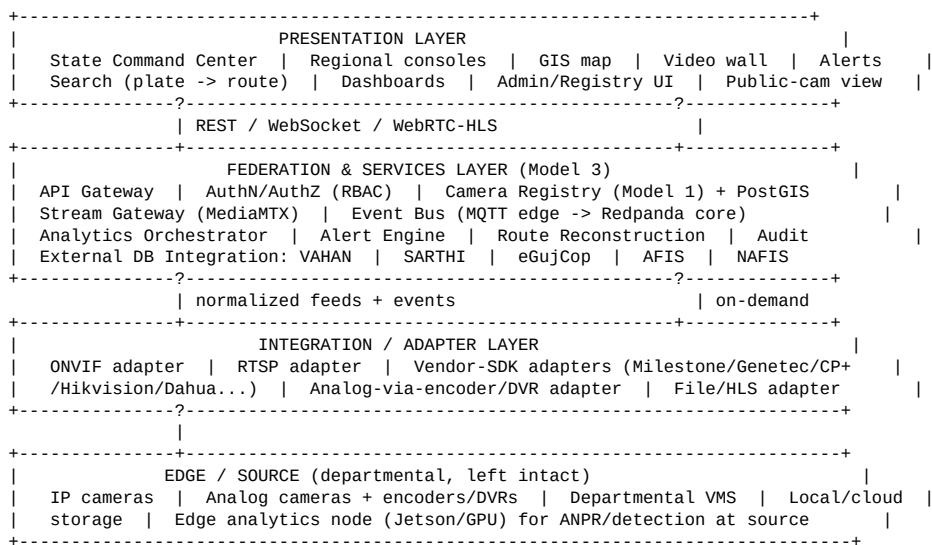
1. **Non-invasive federation** - departments keep their VMS, storage, AMC, and operational control. 2. **Event-driven** - extract intelligence at source; move events, not video. 3. **Vendor-neutral & standards-based** - ONVIF/RTSP first; adapters for proprietary SDKs; documented open APIs; no lock-in. 4. **Hierarchical (edge -> regional -> state)** - bounded WAN traffic, regional autonomy on link loss, scale by adding regions. 5. **Secure & privacy-aware by default** - mTLS, RBAC, network segmentation, tamper-evident audit, consent-gated private cameras. 6. **Prototype ? production interfaces** - the sandbox build and the statewide design share APIs/contracts; only the backing implementations differ. 7. **Data integrity - never fabricate** - cameras come only from the live feed (no synthetic fallback; a failed onboard surfaces the real error). Alerts are computed from real analytics on real feeds, not from invented database records. Derived fields (geocoded coordinates, inferred category) are explicitly flagged, and analytics limitations are documented, not masked.

4. Chosen architecture & rationale

Model	Role in Netra	Why
Model 1 - Registry & GIS (mandatory)	System of record for every camera's metadata, location, health, ownership, connectivity, storage	Compulsory; the visibility/planning foundation all other layers key off
Model 3 - Federation & Middleware	Adapter/connector layer + unified API + stream gateway + event bus between departmental VMS and downstream apps	Answers heterogeneity + scale without touching departments; the interoperability core
Model 4 - Central AI Analytics (selective)	ANPR, detection, cross-camera tracking, DB-integrated alerts	Delivers the scored analytics test case; designed to run at edge/regional scale

We use Model 2's unified-viewing and searchable-metadata capabilities as the operator-facing layer, but place Model 3 adapters behind its stream contract so department/vendor differences do not leak into the viewer. Direct connectors remain possible for the PoC; statewide onboarding uses the federation layer.

5. Logical architecture



Physical/deployment tiers (edge -> regional -> state) and their sizing are in [SCALABILITY.md Section 3](#).

6. Component design

6.1 Camera Registry & GIS (Model 1 - mandatory)

- **System of record** for every camera: id, department, ownership, geo(lat/lng), site, camera_type (IP/analog), make/model, protocol, stream_url, capabilities, connectivity, storage_type, retention_days, health_status, AMC_expiry.
- **Onboarding**: bulk CSV/API import, manual entry, discovery (ONVIF WS-Discovery).
- **GIS**: MapLibre/Leaflet + PostGIS; layers by department, camera type, status, coverage; **gap-analysis** (uncovered zones, ageing/EOL infra, AMC expiry).
- **Health monitoring**: heartbeat/status per camera; RBAC search, filtering, export, audit trail.

6.2 Integration / Adapter layer (heterogeneity)

- **Adapter interface** (connect -> authenticate -> discover -> stream -> status) implemented per source type. Adding a new vendor = a new adapter, no core change (extensible connector framework, FAQ #38).
- **[built]** The **live-feed adapter** onboards the real CSITMS feed over **open HTTP/REST + standard HTTP video** - *no vendor SDK*. It reads `live.corp8.cloud/api/cameras`, maps each camera to the registry schema, sets `stream_url`

= /stream/{id}, geocodes the free-text location to an **approximate** coordinate (flagged coords_approx, since the source API has no lat/long), and infers a coarse category (flagged dept_inferred).

- **[framework-ready]** Additional adapter types the framework supports (not yet built, because the live source doesn't need them): **ONVIF** (Profile S/T/G), **RTSP**, **vendor SDKs** (Milestone, Genetec, CP Plus, Hikvision, Dahua), **analog** via existing encoders/DVR -> RTSP. Onboarding a new source type = one drop-in adapter, no core change.
- Normalizes every source to a common feed descriptor; the stream gateway (Section 6.3) handles container/codec differences.

6.3 Stream gateway [built]

- **[built]** An **ffmpeg-based gateway** (FastAPI service) makes heterogeneous source containers browser-playable without touching the source: **MP4/H.264** is played directly; **MKV** is remuxed (-c copy, cheap); **AVI / unknown codecs** are transcoded to H.264. Output is fragmented MP4 streamed per active viewer and torn down on disconnect. This is the concrete realisation of the "no rip-replace, normalise at the platform edge" principle.
- **Production: MediaMTX** + ffmpeg for RTSP->WebRTC/HLS with session control and on-demand pull (no persistent central pull of all feeds). Same gateway contract.

6.4 Analytics engine (Model 4, selective) [built]

- **[built]** **Two-stage ANPR: YOLOv8n** detects vehicles (type) -> a **dedicated YOLO licence-plate detector** localises plates -> **EasyOCR** reads them (upscaled crop, Indian-plate regex + confidence gate). Overlay bands are masked so the burned-in camera-name/timestamp watermark cannot be mis-read as a plate.
- **[built]** **Vehicle attributes:** dominant **colour** (HSV) + type per vehicle - the key signal for attribute-based tracking when plates are unreadable.
- **[built]** **Intra-camera tracking:** isolated **ByteTrack** state per camera; detections must persist across configurable observations and produce one durable event when the track completes. Type/colour are majority-voted across the track rather than trusted from one frame.
- **[built]** **Verifiable evidence:** the detection retains the vehicle crop; watchlist evidence retains a contextual crop with the triggering track explicitly boxed and labelled.
- **[built]** **Event time:** the source player's wall_time + slot_offset clock is authoritative; burned-in timestamp OCR and explicit file-start configuration are fallbacks. Ingest time is retained separately for throttling and audit.
- **[built]** **Tracking-derived analytics:** image-plane direction, dwell, stopped state and pixel motion are stored per track. Wrong-way is emitted only for cameras with an operator-configured expected direction; road speed remains unavailable until a camera is geometrically calibrated.
- **[built]** **Real finding:** on the current **wide-angle traffic-overview** CSITMS cameras, plates are ~20-120 px (below the OCR floor of ~100 px clean width), so plate *reads* are near-zero on these feeds - a **source-camera resolution** limit, not a pipeline defect. Plate detection works; the pipeline is ready for ANPR-grade (zoomed) cameras in the full 50-camera set.
- **[built]** **Placement (prototype):** central worker, round-robin over an active set, frame-sampled at one synchronized source slot. **Production:** edge/regional GPU pools near the source (see SCALABILITY.md).
- **Model extension point:** VEHICLE_MODEL + VEHICLE_CLASS_MAP_JSON accepts a validated Gujarat-specific model containing an auto_rickshaw class. The stock COCO model does not contain that class and the UI does not pretend otherwise.
- **Roadmap:** validate/fine-tune the auto-rickshaw model, person/face detection, intrusion/loitering/crowd, visual re-ID, and calibrated road speed.

6.5 Event bus [built]

- **[built]** Redis Streams in the prototype (detections published on ingest). **Production: MQTT** edge->regional + **Redpanda** (Kafka API) regional+core, partitioned by region x camera-group, RF=3 - identical producer/consumer interface. Sizing in [SCALABILITY.md Section 2](#).

6.6 Alert engine - real feed analytics [built]

Alerts are computed from **real analytics on the live feeds**, not from any fabricated database:

- **[built] Congestion** - per-camera vehicle count exceeds a threshold.
- **[built] Traffic surge** - count spikes above a live per-camera EMA baseline.
- **[built] Feed-offline** - the health monitor detects a down stream (real: cameras 6 & 22 return HTTP 500) and raises an alert.
- **[built] Operator watchlist (BOLO)** - an operator flags a vehicle (plate, or type + colour); it fires on real detections. Operator-defined, throttled per camera.

6.7 External database integration (deployment)

- Connectors to **VAHAN** (vehicle reg), **SARTHI** (licence), **eGujCop/CCTNS** (arrested/wanted/stolen/missing), **AFIS/NAFIS** (fingerprint), behind an IntegrationConnector interface (caching + rate-limiting). An ANPR plate/face match -> DB lookup -> enriched alert.
- **These are real deployment integrations, not simulated in the prototype.** They activate when (a) the platform is deployed with authorised gov API access and (b) plates/faces are readable (ANPR-grade feeds). The prototype deliberately does **not** fabricate VAHAN/eGujCop records.

6.8 Health monitor & gap-analysis [built]

- **[built] Health monitor** - periodically probes each camera's real stream endpoint (HTTP range request), updates health_status (online/degraded/offline), and raises feed-offline alerts on transition.
- **[built] Gap-analysis** - coverage %, per-district online/offline, thin-coverage districts (≤ 1 camera), and the offline-camera list - from the live registry (Model 1).

6.9 Command center [built]

- **[built]** Single-pane UI: signed login | GIS map (registry) | video wall (all live feeds) | vehicle tracking (search -> route + timeline) | alerts (feed + map) | ops (gap-analysis + audit) | live detections panel | verified RBAC identity and sign-out.

7. Integration strategy for heterogeneous cameras/VMS

Source type	Mechanism	Status
Live CSITMS feed	HTTP/REST + HTTP progressive video (no SDK)	[built]
IP camera (ONVIF)	ONVIF Profile S/T/G + RTSP	[framework-ready]
IP camera (proprietary)	Vendor SDK adapter (Milestone/Genetec/CP+/Hikvision/Dahua)	[framework-ready]
Analog camera	Existing encoder/DVR -> RTSP	[framework-ready]
Departmental VMS	VMS API/SDK adapter or RTSP re-stream	[framework-ready]
Private cameras	Viewing-only adapter, consent-gated	[framework-ready]

Principle: **normalize at the edge of the platform, keep departments untouched**. New source type = new adapter behind a stable interface. The live integration uses **open standards only (HTTP/REST + standard streaming), with zero vendor-SDK dependency** - SDK adapters exist in the framework for departments whose cameras are reachable *only* via a proprietary VMS SDK.

8. Geographic dispersion & connectivity

- **Edge analytics** so intelligence is produced at the source; **event-only backhaul** (~1.6 Gbps peak statewide vs ~200 Gbps of video).
- Ride **GSWAN** where available; SD-WAN/VPN tunnels elsewhere; **store-and-forward** buffering at edge for flaky links.
- Video pulled **on demand** to the center (never a persistent central pull of all feeds); regional storage keeps recent footage local.

- Low-bandwidth strategies: adaptive bitrate, keyframe/thumbnail-first, event-triggered clip upload. Full treatment in [SCALABILITY.md](#).

9. Video analytics approach (detail)

Pipeline [built]: synchronized frame sample -> overlay-mask -> YOLOv8 vehicle detect (+colour) -> per-camera ByteTrack -> dedicated YOLO plate detect -> EasyOCR (upscaled crop, Indian-plate regex + confidence gate) -> track persistence + majority vote -> one `detection_event {track_uuid, camera_id, geo, source_ts, first_seen, last_seen, vehicle_type, colour, plate?, motion, snapshot}` -> publish to bus + index.

Tracking a *single designated vehicle* across a path

The scored test provides a **vehicle number**. Two matching keys, used in priority:

1. **Plate match (exact) [built]**. When the plate is readable, it is a unique key: `GET /api/track?plate=...` pulls all sightings, orders by time, and returns the cross-camera route + timestamped movement history. This is the primary method and is the right answer whenever the feed is ANPR-grade. *On the current wide-angle CSITMS cameras plates are sub-readable, so this path yields little there - but it is exact wherever plates resolve (the full 50-camera set / zoomed cameras).*

2. **Attribute + spatio-temporal disambiguation (for unreadable plates)**. Type + colour alone selects a *class* (all yellow cars), not one vehicle. To isolate a single vehicle we add two constraints:

- **[built] Spatio-temporal gating** (`GET /api/track/vehicle`): from a start sighting, greedily follow to the next-camera sighting that best matches expected travel time, accepting only sightings that are *physically reachable* - elapsed time $\geq$ inter-camera distance / max speed (a vehicle cannot be at camera B before it could drive there), within distance/time caps. Returns the single trajectory with per-hop **gap, distance, and inferred speed**, plus a count of candidates rejected as infeasible. Verified: a "yellow car" reconstructs to a 3-hop Ahmedabad path at 28-58 km/h and correctly refuses to jump to the 330 km-distant Gir Somnath camera.
- **[roadmap] Visual re-identification (re-ID)**: an appearance embedding per vehicle crop, matched across cameras (cosine similarity) under the same space-time gate - sharpens disambiguation when several identical-looking vehicles are space-time-feasible.

Current build status: exact **plate-based** single-vehicle tracking [built]; **class-level attribute** tracking [built] (aggregated per-camera path with sighting counts); **space-time single-vehicle** reconstruction [built]; **re-ID** refinement is the next increment.

Other analytics (roadmap): person/face detection, intrusion/loitering/crowd, anomaly, camera-tamper.

10. Data model (key entities)

- camera (registry, Section 6.1) | department | site | stream_session
- detection_event {id, camera_id, type, plate?, object_class?, confidence, geo, ts, snapshot_ref}
- vehicle_track {plate, [{camera_id, geo, ts, confidence} ...] } (materialized from events)
- watchlist_item | alert {id, rule, event_ref, db_match, severity, status, ts}
- user, role, permission, audit_log

11. APIs (OpenAPI-documented, RBAC-scoped) [built]

Actual endpoints in the prototype (/docs for the live OpenAPI):

- **Registry:** `GET/POST /api/cameras`, `POST /api/cameras/bulk`, `GET /api/stats`, `GET /api/gap-analysis`, `POST /api/ingest/live`, `POST /api/ingest/retry`

- **Analytics:** POST /api/detections, GET /api/detections[/stats], POST /api/frame (per-frame count -> congestion/surge)
- **Tracking:** GET /api/track?plate=|vehicle_type=&color= -> sightings + aggregated per-camera path
- **Alerts:** GET /api/alerts[/stats], POST /api/alerts/{id}/ack, GET/POST/DELETE /api/watchlist
- **Governance:** GET /api/audit, GET /api/health (reports ingest_error)
- **Stream gateway:** GET /gateway/stream/{id}?c=copy|x264 (separate service)
- **Authentication/RBAC:** password login issues an HS256-signed bearer JWT; every protected request derives user, role and district scope from verified claims. The same authorization boundary is ready to accept department SSO/OIDC tokens in deployment. Caller-supplied identity headers are disabled outside an explicit local-development mode.
- **Deployment connectors:** IntegrationConnector interface for VAHAN/SARTHI/eGujCop/AFIS/NAFIS (real, not faked in the prototype).

12. Security, privacy, RBAC & audit

- **[built] AuthZ / RBAC:** role-scoped access - state_admin (all), district_officer (scoped to their district - cameras *and* stats filtered), viewer (read-only; mutations return **403**). Principal carried per request; ready to swap for department SSO/OIDC.
- **[built] Audit:** append-only log of every audited action (vehicle track, alert ack, BOLO add) attributed to user + role + timestamp.
- **[built] Data-minimization:** events + small plate/vehicle snapshots stored, **not** the video (which stays at source).
- **[design] Transport:** mTLS on all feeds/events/APIs; TLS/SRTP for streams.
- **[design] Network:** per-department segmentation; gateway isolation; no lateral access between departments by default.
- **[design] Privacy:** private cameras viewing-only + consent-gated; retention respects departmental policy; face/DB-match features gated + logged; tamper-evident (hash-chained) audit.
- **[design] At rest:** encryption; key management; erasure-coded regional stores.

13. Deployment architecture

Three tiers - **Edge (site)** -> **Regional (district/police range)** -> **State Core (SDC + DR)** - with Kubernetes per region, stateless services behind load balancers, autonomy on WAN loss. Full topology, node counts, GPU/bandwidth/storage sizing, HA/DR: [SCALABILITY.md Section 3-6](#).

14. Scalability to ~80,000 cameras

Summarized headline: video would be ~200 Gbps / ~32 PB centralized -> kept regional; events ~1.6 Gbps peak / ~100 TB (30 d) -> ~0.3% of data; ~600-1,000 distributed accelerators; core event bus at ~5-10% utilization. Model, tiers, storage tiers, HA/DR, and phased rollout: [SCALABILITY.md](#) (evaluation area #35).

15. Department-level technical requirements (integration feasibility)

To onboard each department we need (to be collected via a standard intake form):

Item	Why
Camera inventory (count, IP/analog, make/model)	Adapter selection, sizing
VMS platform + version + API/SDK availability	Federation method
Protocols supported (ONVIF/RTSP/proprietary)	Adapter selection
Network topology, bandwidth, GSWAN/other connectivity	Edge vs central, backhaul plan
Storage type (local/cloud) + retention (7/15/>15 d)	Tiering, on-demand pull design
Existing analytics/encoders	Reuse vs add edge node

Item	Why
Access/security policy, credentials process	mTLS, RBAC, segmentation
AMC vendor + expiry	Coexistence, upgrade path
Site locations (geo)	Registry/GIS, gap analysis

16. Cost-benefit (high level)

- **Reuse-first:** no camera/VMS replacement; reuse encoders, existing storage, GSWAN -> capex avoided.
- **Event-driven** -> ~1.6 Gbps vs ~200 Gbps -> orders-of-magnitude lower WAN/central-storage cost.
- **Selective/edge analytics** -> ~600-1,000 accelerators vs ~2,000 central -> ~50% compute capex saved.
- **Open/vendor-neutral** -> no lock-in; future cameras/departments onboard via adapters, not re-procurement.
- Detailed BOM per regional hub: *to be added* (see Section 18).

17. Test-case walkthrough (scored)

1. Onboard the provided stream URLs via the live adapter -> they appear on the GIS map and video wall. 2. The analytics worker runs vehicle/plate detection + colour across the feeds, emitting indexed events. 3. Judges provide a **vehicle number** -> GET /api/track?plate=... -> **time-ordered cross-camera route + movement-history timeline + map path** (exact, whenever the feed is ANPR-grade). 4. If plates are unreadable on a given feed, attribute + spatio-temporal disambiguation (Section 9) narrows to the single-vehicle trajectory. 5. Congestion/surge/feed-offline and operator-BOLO alerts fire from real analytics; the audit log records every query/action.

18. Assumptions & open items

- Blended stream 2.5 Mbps; frame-sampled analytics; per-GPU camera density to be re-measured on ANPR-grade feeds.
- **[measured]** The live CSITMS feed is HTTP/1.1 progressive MP4/MKV/AVI, 31 cameras, 12-h files starting ~evening, wide-angle overview (plates ~20-120 px).
- **To add:** regional-hub reference BOM (cameras/hub, GPU count, bandwidth, racks, cost); ER + sequence diagrams; single-vehicle re-ID module.
- Real DB (VAHAN/etc.) access is a deployment-time gov integration - **not simulated** in the prototype.

19. As-built summary (what was implemented)

Runs with one command (docker compose up) against the real feed:

Service	Tech	Role
backend	FastAPI + SQLAlchemy + GeoAlchemy2 + PostGIS	registry, analytics ingest, tracking, alerts, RBAC, audit, health
gateway	FastAPI + ffmpeg	MKV/AVI -> browser-playable MP4
analytics	Python worker: YOLOv8 + YOLO plate detector + EasyOCR	detections, colour, ANPR, per-frame counts
frontend	React + Vite + MapLibre	Map / Video Wall / Tracking / Alerts / Ops + detections panel
db / redis	PostgreSQL+PostGIS / Redis Streams	store + event bus

Collected from the live feed (evidence of Model 2-style selective analytics): thousands of detection_events (type + colour + camera + geo + time), camera-wise indexed; searchable movement records via /api/track; event tags (congestion/surge/feed-offline). Video is **not** centrally stored.

19.1 Four-model coverage

Model	Status	What's built
1 - Registry & GIS (mandatory)	? Feature-complete	registry, GIS, bulk/manual/API onboarding, health, maintenance lifecycle, ageing/coverage gaps, RBAC, export + audit
2 - Unified Viewing & Selective Analytics	? Code-complete; proof pending	registered-camera gateway, video wall, tracked evidence, indexing, search, alerts and ANPR consensus; official proof needs a real second source and a readable plate sequence
3 - VMS Federation & Middleware	? Code-complete; proof pending	CSITMS + configurable RTSP adapters, canonical provenance, gateway, Redis event bus, correlation dashboard/report; official two-system proof needs a real second VMS/RTSP feed
4 - Central VMS & AI Platform	? Partial	AI analytics + tracking + integration-readiness + RBAC built; tiered storage / DR / GPU-pool / 80k scale is design

19.2 Architecture-principles compliance

Principle	Realisation
Open vendor-neutral no lock-in	100% open-source; live integration over HTTP/REST + standard streaming, no vendor SDK
Modular technology-agnostic	services behind stable contracts; ANPR model, event bus, storage, auth all swappable (ANPR model is an env var)
Standards-based	REST/OpenAPI, PostGIS (OGC), ONVIF/RTSP/HLS-ready, Kafka-API/MQTT (design)
Heterogeneous multi-vendor	adapter layer + ffmpeg gateway normalise mixed protocols/containers/codecs
Upgrade/replace/expand w/o redesign	proven in-build: swapped camera source via one adapter; added a whole analytics stage + new alert types without touching core contracts
Secure	RBAC + audit built; mTLS/segmentation/encryption designed
Scalable	event-driven, hierarchical edge->regional->state (SCALABILITY.md)

19.3 Honest limitations (documented, not masked)

- **ANPR plate reads** are near-zero on the current wide-angle overview cameras (plates below OCR resolution) - a source-camera limit; pipeline is ready for ANPR-grade feeds.
- **Single-vehicle tracking** without a plate is built via space-time gating (Section 9); a **visual re-ID** refinement (to separate identical-looking, space-time-feasible vehicles) is the next increment.
- **Coordinates** are geocoded/approximate (source API has no lat/lng), flagged coords_approx.
- **Scale/DR/storage** is design, appropriate for Phase-1.

*Companion docs: [PLAN.md](#) (master plan), [SCALABILITY.md](#) (80k-camera sizing). Sections tagged **[built]** are implemented in the running prototype; **[design]/[framework-ready]/[roadmap]** are specified for deployment.*

Scalability and Capacity Plan

Statewide Scalability Design - ~80,000 Cameras

> Deliverable for evaluation area #35 (statewide scalability plan). Feeds directly into the HLD. > All figures use stated assumptions; ranges given where inputs vary. Round numbers are deliberate - the point is the order of magnitude, which drives the architecture.

1. Capacity model (why the architecture is what it is)

Assumptions: blended stream ? **2.5 Mbps** (mix of 2 MP @ 2 Mbps H.265, 4 MP @ 4-6 Mbps, and low-bitrate analog-encoded). 80,000 cameras.

1.1 Video bandwidth - the reason you cannot centralize

Metric	Value
Per camera	2.5 Mbps
Aggregate if all video centralized	80,000 x 2.5 Mbps = ~200 Gbps

200 Gbps of sustained ingest into one core is infeasible and pointless. -> **Video stays at edge/regional. Only events + on-demand streams cross the WAN.** This is the architectural thesis, forced by arithmetic.

1.2 Video storage (kept regionally, tiered)

Metric	Value
Per camera/day	2.5 Mbps ? 27 GB/day
All cameras/day	~2.16 PB/day
15-day retention	~32 PB (before codec savings)
With H.265 + motion/event-based recording	40-60% less -> ~13-19 PB

-> **Distributed regional storage**, tiered hot/warm/cold. Never a single central store.

1.3 Analytics compute (GPUs)

Assumptions: analytics at **5 fps** sampling (not 25-30); full ANPR pipeline (plate detect + OCR + vehicle detect) ? **~40 cameras per modern GPU** (L4/A10-class); lighter motion/object-only ? 80-120/GPU.

Scenario	GPU estimate
Worst case: continuous ANPR on all 80k	80,000 / 40 ? ~2,000 GPUs
Realistic: ANPR on ~30-40% (public/traffic domain), rest motion-triggered + edge accelerators (Jetson Orin at camera clusters)	~600-1,000 accelerators, regionally distributed

-> **Edge + regional inference**, selective/motion-triggered analytics, frame sampling, ROI. Not central GPU farm.

1.4 Event bus - the punchline

Assumptions: avg **0.5 events/sec/camera** (bursty; ANPR/detection only fire on activity), payload ~1 KB.

Metric	Value
Avg event rate	80,000 x 0.5 = ~40,000 events/sec
Peak	~150,000-200,000 events/sec
Peak throughput	~200k/s x 1 KB = ~200 MB/s (~1.6 Gbps)

Metric	Value
Event log/day (avg)	~3.4 TB/day
30-day event retention	~100 TB

Compare: events ? **100 TB** vs video ? **32 PB** -> events are ~0.3% of the data. A modern Kafka/Redpanda broker sustains 100s of MB/s-GB/s each; a **6-12 node core cluster handles millions/sec** - our 40k-200k/s is comfortable headroom. **The bus is never the bottleneck; that's exactly why the system is event-driven.**

2. Messaging tier at scale (the Kafka-alternative decision, sized)

Two-tier, matched to the 1,000 km dispersion:

- **Edge -> Regional: MQTT** (EMQX/Mosquitto cluster per region). Lightweight, tolerant of thin/flaky links, ideal for pushing *events* (never video) from edge analytics nodes. Directly answers "geographical dispersion" + "low-bandwidth strategy."
- **Regional core + State core: Redpanda** (Kafka API, single binary, no ZooKeeper/JVM - far simpler gov ops) *or* NATS JetStream (lightest). RF=3, partitioned by region x camera-group.
- **Bridge:** MQTT -> Redpanda connector at each regional hub aggregates/filters before anything crosses to the state core.
- **Prototype uses Redis Streams** (identical producer/consumer interface) so hackathon code is unchanged; HLD shows the MQTT+Redpanda production topology.

Sizing: per-region Redpanda 3-node (RF3) handles that region's slice; state core 6-12 nodes for cross-region correlation + forensic event log (30-90 day retention for replay). All well within single-cluster limits.

3. Hierarchical topology (edge -> regional -> state)

TIER 0 - EDGE (per site / camera cluster)
Existing camera + NVR/VMS (untouched) + Edge analytics node (Jetson/GPU)
-> ANPR/detection at source | MQTT event publish | local buffer for flaky links

TIER 1 - REGIONAL (per district/police range - ~6 ranges / 33 districts)
Regional VMS federation gateway (adapters) | Regional storage (hot+warm, tiered)
Regional GPU analytics pool | MQTT broker + Redpanda cluster | regional command view
-> keeps video local; publishes filtered/aggregated events upward

TIER 2 - STATE CORE (State Data Center, Gandhinagar + DR site)
Global camera Registry + PostGIS | Global event store + search index (ES/Timescale)
Cross-region correlation & vehicle route reconstruction | State Command Center
External DB integration: VAHAN | SARTHI | eGujCop | AFIS | NAFIS
-> sees events + metadata + on-demand streams, never bulk video

Why hierarchical: bounds WAN traffic (events only), keeps video/storage/compute near the source, lets regions run degraded-independent if the WAN drops, and scales by adding regions - not by growing one core.

4. Cross-camera vehicle tracking at 80k scale (the test case, scaled)

1. Edge/regional analytics emits {plate, camera_id, geo, ts, confidence} on every ANPR hit -> MQTT -> Redpanda. 2. A stateful stream processor (Kafka Streams / Flink, or a consumer group) maintains per-plate timelines and writes to an **indexed event store** (Elasticsearch or TimescaleDB + PostGIS). 3. **Route query = an indexed search over events**, not video: given a plate, pull ordered (geo, ts) across all regions -> reconstruct route + timestamped movement history in ms. 4. Fuzzy plate matching (OCR confidence + Levenshtein) + optional visual re-ID as corroboration.

At scale this is a search problem over ~100 TB of tiny events - fast and horizontally shardable - **not** a 32 PB video problem.

5. Storage tiers (retention-driven)

Tier	Window	Media	Location
Hot	0-48 h	NVMe	Regional
Warm	2-15 d	HDD / object store	Regional

Tier	Window	Media	Location
Cold / archive	>15 d (policy-based)	Erasure-coded object / archive	Regional or state DR
Events & metadata	30-90 d+	Indexed store	State core (retained far longer than video - cheap)

6. HA / DR / security at scale

- Redpanda RF=3; PostGIS + event store replicated to DR site; regional autonomy on WAN loss.
- Horizontal scaling: add regions/brokers/GPU pools; stateless services behind load balancers; Kubernetes per region.
- Health/monitoring: per-camera heartbeat, Prometheus/Grafana, gap-analysis (Model 1).
- Security: mTLS on all feeds/events, network segmentation per department, RBAC, tamper-evident audit log, encryption at rest.

7. Phased statewide rollout

1. **Pilot** - 2-3 districts, ~2,000 cameras, one regional hub. Validate onboarding, ANPR, cross-camera tracking, HA. 2. **Range expansion** - scale to a full police range; tune GPU/bandwidth ratios against real load. 3. **Statewide** - replicate the regional template across all ranges/districts; connect DB integrations; DR cutover drills. 4. **Private-camera onboarding** - societies/malls (viewing-only, consent-gated) where permitted.

8. Indicative cost envelope

These are planning ranges, not vendor quotations. They exclude the existing 80,000 cameras/NVRs and land/buildings, and should be replaced by discovered department inventory plus RFP pricing. The transparent model is more defensible than a single unsupported total.

Cost block	Planning basis	Indicative range
Distributed analytics	600-1,000 blended edge/regional accelerators at INR 4-10 lakh	INR 24-100 crore
13-19 PB usable tiered storage	INR 12,000-20,000 per usable TB including controllers/replication	INR 156-380 crore
Regional/core compute, event bus, WAN uplift, HA + DR	Six range hubs plus state and DR cores	INR 80-160 crore
Platform engineering, adapters, cybersecurity, integration and rollout	Multi-department staged implementation	INR 100-200 crore
Statewide capital envelope	Existing camera estate reused	INR 360-840 crore
Annual operating envelope	Support, bandwidth uplift, power/DC, replacements, security operations	INR 55-140 crore/year

Pilot gate: budget **INR 10-18 crore capital** and **INR 2-4 crore/year operating** for ~2,000 cameras across 2-3 districts, then replace every assumption with measured bitrates, retention policy, accelerator throughput and discovered infrastructure before statewide procurement.

9. Headline numbers (for the deck)

- ~200 Gbps video if centralized -> **so we don't**; events are ~1.6 Gbps peak.
- Video ~32 PB @ 15 days vs events ~100 TB @ 30 days -> events are ~0.3% of the data.
- ~600-1,000 distributed accelerators (realistic) vs 2,000 (naive central).
- Core event bus runs at ~5-10% of a modest cluster's capacity -> massive headroom.

API and Integration Contract

Netra Registry and Federation API

The interactive OpenAPI specification is available at <http://localhost:8000/docs>. Use `POST /api/auth/login` to obtain a signed JWT and send it as `Authorization: Bearer <token>`. Identity, role, and district scope are taken only from verified token claims; write operations require `state_admin` or an in-scope `district_officer`. `GET /api/auth/me` returns the verified current identity.

Model 1 - registry and asset lifecycle

- `GET /api/cameras` - filter by department, status, type, city, analytics flag, source system or free-text query.
- `POST /api/cameras` - manually onboard one camera.
- `POST /api/cameras/bulk` - onboard a validated JSON/CSV-derived batch.
- `PATCH /api/cameras/{camera_id}/lifecycle` - update installation, service, maintenance, end-of-life and notes fields.
- `GET /api/stats` - registry totals grouped by department, status, type, city and source system.
- `GET /api/gap-analysis` - coverage, offline feeds, thin districts, maintenance due, EOL assets and metadata completeness.
- `GET /api/reports/cameras.csv` - audited, spreadsheet-safe registry export.

Model 2 - viewing and searchable analytics

- `GET /gateway/camera/{camera_id}/snapshot` - codec-independent JPEG from a backend-registered source URL.
- `GET /gateway/camera/{camera_id}/stream?c=copy|x264` - on-demand fragmented MP4 relay/remux/transcode; the gateway never accepts arbitrary URLs.
- `GET /api/detections` and `/api/detections/stats` - evidence-backed tracked events only.
- `GET /api/track` and `/api/track/vehicle` - plate or attribute movement history and space-time-gated route reconstruction.
- `GET /api/reports/detections.csv` - timestamped movement-evidence export.

Model 3 - adapters and federation

- `GET /api/adapters` - configured source adapters and persisted coverage.
- `POST /api/ingest/adapters` - discover all configured systems independently; one unreachable system does not hide healthy systems.
- `GET /api/reports/federation` - per-system cameras, online feeds, analytics coverage and events. `federated=true` only when at least two non-manual, adapter-backed source systems exist.

Configure independent RTSP/ONVIF-style sources with `RTSP_SOURCES_JSON`; see `.env.example`. Source credentials remain server-side and are never exposed as gateway query parameters.

Requirements Traceability

Official Requirements -> Netra Execution Plan

Audit of the Gujarat CCTV Hackathon 2026 problem statement on 21 Aug 2026. This file separates working evidence from roadmap claims.

Official requirement	Working evidence in this repository	Status / next action
Open, modular, vendor-neutral integration	Live-feed adapter, normalized camera schema, HTTP/RTSP/HLS/ONVIF metadata, ffmpeg gateway	Working; publish adapter interface/API examples
Mandatory Model 1 registry + GIS	Map plus Registry workspace; search/filter; manual and CSV onboarding; health, provenance and analytics fields	Working
Role-based search, actions, export and audit	Signed JWT identity, state/district/viewer principals; district scoping; actor-gated camera writes; audited CSV exports	Working prototype; integrate department SSO/OIDC for deployment
Bulk and manual camera onboarding	POST /api/cameras, /bulk, Registry form and CSV import	Working
Health, maintenance and coverage gaps	Stream health monitor, offline alerts, Ops gap report	Working; add maintenance tickets/AMC workflow
Search designated registration number	Exact/normalized ANPR tracking endpoint and Tracking UI	Working when source imagery yields a readable plate
Complete route and timestamp/location history	GIS route, ordered timeline, CSV movement-evidence export	Working
Representative watchlist + continuous alerts	Track-persistence/confidence-gated matching, explicitly boxed context evidence, source-clock event time, live alert UI	Working; validate thresholds per camera/model
Own-feed 2-3 minute demonstration	Independent RTSP onboarding and guided runbook exist	Record the final genuine-road video
Government-feed analytics output report	CSV contains detection, plate, timestamp, camera, district, coordinates and evidence path	Working; improve ANPR yield on wide-angle feeds
~50 heterogeneous feeds on event day	30 live government feeds normalize/play; ingestion benchmark exceeds the 250 frame-summary/s target	Execute the final supplied-feed timed rehearsal
80,000-camera architecture, sizing, HA/DR, costs	SCALABILITY.md includes bandwidth, storage, GPUs, tiering, HA/DR, rollout and cost envelope	Documented; validate assumptions during discovery
Submission artefacts	README, HLD, scalability, API document, deck and traceability matrix	Export/verify PDFs, record two timed videos, host and issue evaluator credentials

Execution order from here

1. Record genuine multi-camera ANPR-grade road footage so judges can see onboarding -> analytics -> route without depending on remote stream conditions. 2. Raise ANPR evidence quality: plate ROI diagnostics, confidence calibration, Indian-plate benchmark set, and camera suitability labels. 3. Run a 50-feed rehearsal and capture latency, throughput, failure recovery and per-stage timings as evaluation evidence. 4. Integrate department OIDC/SSO, signed audit records, secrets management and a least-privilege deployment profile. 5. Produce the timed videos, HLD/API appendix, deck, hosted demo and credentials.